Thuyết minh sản phẩm ViBioMIR

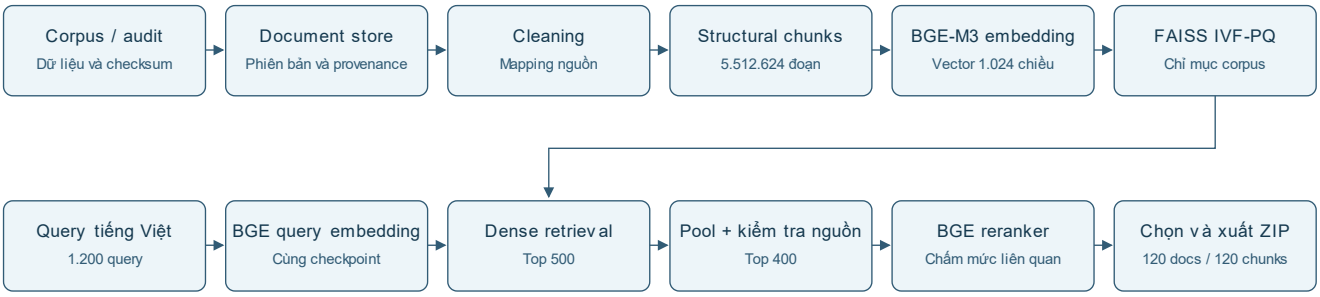
Đội: onlyrelu. **Người đại diện:** Võ Trần Duy Hoàng. **Ngày:** 04/11/2026. **Bài private được đội xác nhận:** submission_17.zip.

1. Mục tiêu và phạm vi

Hệ thống nhận câu hỏi tiếng Việt và trả về danh sách tài liệu cùng đoạn evidence y sinh liên quan từ corpus đa ngữ Việt–Anh–Trung. Đầu ra là dự đoán truy hồi, không phải câu trả lời tư vấn y khoa được sinh bởi LLM. Văn bản evidence được giữ theo dữ liệu nguồn.

Danh tính đội, link bàn giao và bài private test cuối cùng được khai báo trong LINKS.json và FINAL_SUBMISSION.json cùng thư mục. Hash submission_17.zip và submission.json khớp REPORT của export dense V1 trên Drive, xác minh ngày 04/11/2026; EXPERIMENT.json xác nhận dense-only cross-encoder, top400 và selection 120/120/6/0,8. Bản sao ba JSON nguồn nằm trong evidence/. Hồ sơ không giả định rằng các nhánh Qwen/E5/fallback góp phần vào bài private này.

2. Kiến trúc tổng thể



Sơ đồ mô tả đường dense V1 hoàn chỉnh đã được chạy và dùng làm cơ sở tái hiện. Ngoài ra repository có baseline hybrid BM25+BGE, lựa chọn lại evidence bằng Qwen/BGE và nhánh partial fallback; các khác biệt được trình bày ở phần 6.

3. Tiền xử lý và chunking

Audit kiểm tra shard, schema, ID, văn bản, trùng lặp/phiên bản và các dấu hiệu lỗi trích xuất. Document store giữ nhiều version của cùng document cùng checksum và provenance, thay vì ghi đè nguồn cũ.

Bước cleaning dùng quy tắc được review để xử lý control/footer/feed và tận dụng block/HTML local khi có thể khôi phục. Các trường hợp không đủ bằng chứng được giữ trạng thái chưa phục hồi hoặc cần kiểm tra; không tự viết lại nội dung y khoa. Mỗi phần cắt bỏ có vị trí nguồn và revision rule. Những đặc điểm như số, đơn vị và phủ định được bảo vệ bởi các điều kiện veto trong rule.

Structural chunking chia theo cấu trúc section/đoạn/câu và sử dụng token-offset window cho phần quá dài. Không nối qua khoảng nguồn đã bị loại. Tham số cấu hình gốc: target 384 token, max 512, overlap 64, min 128, context 48; các ràng buộc cấu trúc và phần ngắn có thể làm kích thước thực khác target.

chunk_text là evidence nguyên văn. retrieval_text là biểu diễn phục vụ tìm kiếm, có thể thêm context/title từ nguồn. Mapping lưu document ID, derived version, region, offsets, batch/row/chunk order và hash văn bản.

Snapshot structural đầy đủ: **855.745 document / 5.512.624 chunk / 1.712 batch**. Corpus semantic 9.036.279 chunk là artifact khác, không thay thế tương đương snapshot structural.

4. Xây chỉ mục và truy hồi dense

BGE-M3 được khóa revision, output 1.024 chiều. Dữ liệu dài được chia window phủ nội dung thay vì cắt bỏ token vượt giới hạn. Mỗi window dùng CLS pooling và L2 normalization; vector chunk được gộp theo trọng số số token rồi L2 normalize lại. Vector lưu float16; phép gộp/chấm dùng float32. Chính sách window cụ thể của Stage 8 được ghi trong plan/config, không thay bằng policy của notebook embedding thử nghiệm mới.

Các source batch được giao cho sáu logical worker độc lập. Output từng batch gồm vector, mapping và manifest; khi merge phải đủ batch và khớp source/model/plan.

Chỉ mục baseline dùng FAISS IVF-PQ: nlist=4096, m=64, nbits=8, nprobe=32, tối đa 200.000 vector corpus làm mẫu train và seed=270126. Train ở đây là lượng tử hóa chỉ mục, không fine-tune model và không dùng nhãn query. Code có thể giảm số centroid/bit cho corpus nhỏ; manifest ghi thông số thực.

Query dùng cùng encoder/pooling policy với corpus của nhánh. Dense V1 trả tối đa 500 candidate/query kèm chunk ID, source mapping và score. Chỉ mục approximate có thể bỏ sót candidate; việc xây index thành công không phải chứng nhận recall theo GT.

5. Rerank và chọn đầu ra

Đường dense recovery nhận top 400 candidate/query từ Dense V1, đọc lại source text và kiểm tra chunk/document/version/hash. BGE reranker-v2-m3 chấm query–retrieval_text. Frozen S09 worker hỗ trợ window input dài và cache theo đúng query/passage/policy; không reuse score chỉ vì chunk ID giống nhau.

Các tham số đầu ra của đường dense recovery:

Tham số	Giá trị
Candidate Dense V1	Tối đa 500/query
Pool rerank	Tối đa 400/query
Document xuất	Tối đa 120/query
Chunk xuất	Tối đa 120/query

Tham số	Giá trị
Chunk/document	Tối đa 6
Ngưỡng loại chồng lấn nguồn	IoU 0,8

Document được xếp dựa trên score chunk cao nhất. Evidence được chọn trong giới hạn toàn query/per-document, loại exact duplicate trong document, giữ một derived version/document và loại overlap lớn trong cùng vùng. Một số document có thể không còn chunk sau giới hạn toàn query; mọi chunk xuất vẫn phải có parent document trong relevant_docs.

Schema đầu ra:

```
[
  {
    "id": 1,
    "relevant_docs": [123],
    "relevant_chunks": [
      {"doc_id": 123, "chunk_text": "Đoạn evidence nguyên văn từ nguồn.", "chunk_order": 0}
    ]
  }
]
```

Ví dụ chỉ minh họa schema, không phải đáp án của query 1. chunk_order ở submission là thứ tự xuất, không đại diện offset nguồn. ZIP chỉ chứa submission.json ở thư mục gốc.

6. Các phiên bản triển khai và trạng thái

Luồng	Nội dung	Trạng thái bằng chứng
Hybrid S07–S09 cũ	BM25 + BGE, RRF, BGE rerank	Có artifact hoàn chỉnh và validation trong run registry; config gốc top200, 20docs/40chunks khác dense recovery
Dense V1 recovery	Candidate BGE top500, rerank top400, 120docs/120chunks	Đã chạy theo báo cáo người vận hành; điểm 0,1706 do người vận hành cung cấp, chưa có receipt gắn hash trong hồ sơ hiện tại
Multimodel trên submission	Qwen/BGE rerank evidence trong ZIP, gộp RRF	Đã có RRF_80_cap2.zip local và audit; chưa có điểm mới được xác nhận
Corpus retrieval mới	BM25 + BGE + Qwen + E5 từ corpus	Các log được cung cấp cho thấy chưa hoàn tất; không trình bày là full run đã được chấm
Partial retrieval fallback	Snapshot candidate đã quét + evidence baseline; một reranker chấm chung; query chưa xử lý giữ baseline	Có code/notebook và kiểm thử offline; file cuối cùng cần xác nhận qua FINAL_SUBMISSION.json

Partial fallback không coi thiếu một nhánh là có đủ corpus. Mỗi checkpoint được kiểm tra identity/coverage và copy thành snapshot bất biến. Candidate mới được hydrate từ source; baseline evidence chỉ được xác nhận từ ZIP. Cả hai chấm bằng chunk_text để thống nhất đầu vào. Mặc định giữ

relevant_docs cũ; UPDATE_DOCUMENTS=True cho phép xếp lại/thêm document. Score lỗi hoặc thiếu khiến toàn bộ query dùng baseline.

7. Model, dữ liệu huấn luyện và API

Các model inference tải từ checkpoint công khai đã khóa revision; không có fine-tuning neural model trong các đường nêu trên. Không có closed-model API trong đường inference được mô tả. Prompt/instruction Qwen là cấu hình reranker/embedding, không dùng để sinh hoặc sửa evidence. Danh mục checkpoint và cách tải nằm trong MO_HINH.md.

Repository có công cụ silver/proxy và review mẫu phục vụ phát triển, nhưng chưa có GT được xác nhận để khẳng định chất lượng toàn corpus. Các ví dụ do assistant soạn không được trình bày là nhãn BTC hoặc GT đã review.

8. Kết quả và giới hạn đánh giá

Người vận hành đã cung cấp bộ metric sau cho một lần dense recovery:

Metric	Giá trị được báo cáo
FINAL_SCORE	0,1706
DOCS_F2MACRO	0,2534
CHUNKS_F2MACRO	0,0877
DOCS_PRECISION	0,1842
DOCS_RECALL	0,3336
CHUNKS_PRECISION	0,0669
CHUNKS_RECALL	0,1160

Đây không phải điểm private cuối cùng mặc định. Cần ghi đúng submission ID/receipt/hash trong FINAL_SUBMISSION.json nếu báo cáo nó như kết quả bài nộp cuối.

Theo thông báo BTC được đội cung cấp, chunk match cần đồng thời LCS/độ dài đáp án ≥0,4 và LCS/độ dài dự đoán ≥0,4. Hồ sơ không có evaluator và nhãn chính thức để tự xác nhận hai điều kiện này. IoU 0,8 giữa prediction là quy tắc loại trùng nguồn, khác với LCS 0,4 khi so đáp án.

Validation trong code đo integrity, coverage, source mapping, schema và allowlist, không đo độ đúng lâm sàng. Baseline/fallback không tự bảo đảm model đồng thuận là đúng. Thử nghiệm chưa hoàn tất không được gán điểm của một pipeline khác.

9. Truy xuất mã nguồn và tái hiện

Mã chính: vibiomir/ cho audit nối tiếp, document store, cleaning, chunking, lexical/dense index, S09; scripts/ cho adapter, notebook builders, fallback và phân tích artifact; colab/ cho workflow GPU; configs/

và requirements-stage*.txt cho cấu hình/phụ thuộc. Run registry và reports giữ bằng chứng kỹ thuật của các lần trước.

TAI_HIEN.md mô tả cả đường khôi phục từ artifact và đường xây từ dữ liệu gốc. Model/config/source/packet hashes bảo vệ việc resume. Không sửa manifest để làm một artifact khác nguồn trở thành hợp lệ. Để đối chiếu bài nộp theo byte, cần cùng prepared packet và score đã lưu; rebuild GPU từ đầu chỉ kỳ vọng tương đương quy trình, không cam kết bitwise trên phần cứng khác.

Dữ liệu, nguồn gốc và artifact bàn giao

1. Nguồn và quyền truy cập

Nguồn bài toán là ViBioMIR do AIGuruTinix công bố. Corpus làm việc trong repository gồm các shard đã được thu thập tại các domain nguồn, metadata, block và payload HTML local khi có. Các file raw đang dùng phải được chia sẻ qua link đội quản lý trong LINKS.json; link dataset công khai không thay thế cho đúng snapshot shard đã xử lý của đội.

Đợt xử lý được mô tả sử dụng dữ liệu đã có và khôi phục offline từ block/HTML đã lưu; không khẳng định đã thu thập lại toàn bộ Internet hoặc toàn bộ URL official. Official links là bảng đối chiếu ID; không được dùng thay corpus nội dung. Query chính thức là đầu vào inference, không phải nhãn huấn luyện.

Dataset card và nguồn gốc nội dung vẫn cần được giữ khi chia sẻ. Hồ sơ không tuyên bố đội sở hữu bản quyền văn bản của các domain gốc. Chỉ chia sẻ theo quyền và điều kiện BTC/dataset cho phép; phạm vi dữ liệu bên ngoài chưa có trong route được chọn không được gộp ngầm vào artifact.

2. Quy mô và xử lý

Giai đoạn	Số liệu lưu trong báo cáo repository
Raw audit	1.195 shard / 1.105.160 record
Document store	913.969 version / 855.745 document ID
Cleaning đầy đủ	1.029.307 region; 147.803 lượt loại bỏ theo rule đã review
Structural store	5.512.624 chunk / 1.712 batch
Semantic store riêng	9.036.279 chunk; không dùng thay structural
Query chính thức	1.200 query

Không suy ra mọi record là một document độc lập hoặc một ngôn ngữ đã xác nhận. Một document có thể có nhiều version; heuristic language/script không phải nhãn ngôn ngữ đã được con người kiểm duyệt.

Chuỗi xử lý: audit checksum/schema → import version/provenance → cleaning có vị trí nguồn → chọn derived version theo policy → tạo region/chunk → kiểm tra text/hash/offset → embedding/index/candidate. Rule cleaning chi tiết tại configs/cleaning_stage4_reviewed.json; cấu hình chunk tại configs/chunking_stage5.json.

Evidence không được sửa chính tả, dịch hoặc diễn giải lại khi xuất. Chuẩn hóa phục vụ analyzer/vector là biểu diễn tìm kiếm riêng, không thay chunk_text gốc.

3. Snapshot chuẩn


```
{
  "signature": "5ce3d7c02e6a0a8275e203a4cff9c12fb371fc8cf0fd9515574250acfff3751c",
  "manifest_sha256": "d88fd1c046a93950c7e97ef5c8786d933a677ef1f54e7880f5c34fa6c4ff40f1",
  "validation_sha256": "5bdc88900f47989a4edb653ae1f6b90ba90a79760f620edb35ac09a48d20d993",
  "documents": 855745,
  "chunks": 5512624,
  "batches": 1712
}
```


Query SHA256: `ce13508b893bbd6b9169679d540630bc37eb5950cad08b4d5e0e5d7063670c08`.

Official links SHA256: `87e44a792d7a99fb0fde5dff7f09f1a38a0a1fdf02ddd1ac93a005736e00b0a7`.

Sáu ZIP Stage 8 hợp lại chứa source structural. Mỗi ZIP chỉ có một phần batch, cùng plan và metadata; không coi một ZIP là corpus đầy đủ. Kiểm tra bằng stage9_dense_recovery.inspect_bundles và checksum payload từng batch. Dữ liệu cùng số batch nhưng khác signature/chunk count không tương thích.

4. Bố trí dữ liệu trên Drive cho dense recovery

```
MyDrive/ViBioMIR/
  Data.zip
  Data/official/ViBioMIR/
    query.parquet
    links_corpus_vi_en_zh.parquet
  stage9_rerank_colab.zip
  dense_stage8_bge_structure/
    bundles/worker_0.zip ... worker_5.zip
    query_results_v1/results/
      manifest.json
      CORPUS_COVERAGE.json
      candidates/query-1.json ... query-1200.json
```

Data.zip có thể thay file official đã giải nén khi notebook tìm được duy nhất đúng thành viên. stage9_rerank_colab.zip là code/packet S09 có BUNDLE.json, không phải weights. Không dùng artifacts.zip hiện có như structural store nếu nó chỉ chứa cleaning database.

Để dựng từ đầu cần thêm raw shards, audit input_manifest/scan_status/record_audit, document store/cleaning hoặc đủ nguyên liệu tạo lại chúng, tokenizer registry và output embedding/index nếu muốn bỏ qua bước encode. Các file quá lớn không nằm trong ZIP mã nguồn, phải có tại link Drive.

5. Artifact phụ thuộc route cuối cùng

Route	Ngoài source/query/links cần bàn giao
dense_v1_bge	Dense V1 candidates, frozen S09 bundle, prepared.zip và worker score/manifest của run dense recovery nếu cần tái hiện đúng bytes

Route	Ngoài source/query/links cần bàn giao
submission_multimodel_selection	Baseline ZIP thật, run plan/generation policy, score Qwen/BGE đủ query và REPORT/provenance của lần merge
partial_retrieval_fallback	Baseline ZIP, frozen snapshot JSON và PRODUCER_PLAN, fallback PLAN, prepared packets, score đã hoàn tất, QUERY_AUDIT/REPORT của lần export

Với fallback, chỉ những query đã chấm hợp lệ mới đổi. Vì vậy bàn giao đúng tập score tại thời điểm export là thiết yếu; chạy chấm thêm query rồi export lại có thể tạo bài khác. Cần giữ checkpoint/export gốc bất biến.

Các vector embedding mới dang dở vẫn hữu ích để resume, nhưng không phải điều kiện để BTC kiểm tra lại lựa chọn từ snapshot candidate đã đóng băng. Database BM25 local chưa backup cần được bảo toàn riêng trước khi reset runtime; không coi log “1712 batches” là bằng chứng đã có đủ candidate query.

6. Link và kiểm tra bàn giao

LINKS.json là nơi khai báo link Drive/OneDrive thực tế. Chủ đội cần kiểm tra bằng một tài khoản khác quyền tải các artifact bắt buộc, không chỉ quyền xem thư mục trống; xác nhận tên file/path khớp hướng dẫn. Công cụ đóng gói chỉ kiểm tra trường URL và cờ xác nhận, không tự chứng thực quyền truy cập tài khoản BTC.

Đặt FINAL_SUBMISSION.json và REPORT/PLAN của run cuối trong cùng thư mục Drive với ZIP private test để BTC đối chiếu. Không chia sẻ token HF, cookie, credential hoặc thư mục môi trường cá nhân.

Model, checkpoint và cấu hình inference

1. Danh mục và phạm vi sử dụng

Vai trò	Model	Revision khóa	Phạm vi
Tokenizer/chunking và dense baseline	BAAI/bge-m3	5617a9f61b028005a4858fdac845db406aefb181	Structural + Dense V1
Reranker baseline	BAAI/bge-reranker-v2-m3	953dc6f6f85a1b2dbfca4c34a2796e7dde08d41e	S09, dense recovery, fallback BGE
Reranker Qwen	Qwen/Qwen3-Reranker-0.6B	6e9e69830b95c52b5fd889b7690dda3329508de3	Chấm lại evidence hoặc fallback Qwen nếu route chọn
Dense Qwen	Qwen/Qwen3-Embedding-0.6B	97b0c614be4d77ee51c0cef4e5f07c00f9eb65b3	Retrieval đa nhánh thử nghiệm, chưa hoàn tất theo log
Dense E5	intfloat/multilingual-e5-large	3d7cfbdacd47fdda877c5cd8a79fbcc4f2a574f3	Retrieval đa nhánh thử nghiệm, chưa hoàn tất theo log

Qwen3-Reranker-4B tại revision `22e683669bc0f0bd69640a1354a6d0aebcfeede5` được hỗ trợ làm arbiter tùy chọn trong notebook multimodel; không mặc định coi là đã chạy hoặc là thành phần bài private. mBERT revision `3f076fdb1ab68d5b2880cb87a0886f315b8146f8` có trong code control/silver, không phải encoder Dense V1 được mô tả.

Không fine-tune các neural checkpoint này trong các route bàn giao. Giới hạn parameter và revision được kiểm tra trong loader/registry. Ngày model/revision lưu trong policy; danh sách trên mô tả đúng checkpoint đã khóa, không dùng nhánh main tự cập nhật. Các model nêu chính có revision trước mốc 01/08/2026 theo metadata đã kiểm tra trong quá trình phát triển; BTC có thể kiểm tra lịch sử public checkpoint.

2. Link tải chính thức

- BGE-M3 revision dùng cho baseline.
- BGE reranker revision dùng cho baseline.
- Qwen reranker 0.6B.
- Qwen embedding 0.6B.
- Multilingual E5 large.
- Qwen arbiter 4B tùy chọn.
- mBERT control.

Không cần đính kèm weights vào ZIP mã nguồn nếu BTC tải được các đường dẫn công khai này. Không có API key đóng gói trong hồ sơ.

3. Cách tải và sử dụng

Ví dụ tải nguyên checkpoint, không đổi revision:

```
from huggingface_hub import snapshot_download
snapshot_download(
    repo_id='BAAI/bge-m3',
    revision='5617a9f61b028005a4858fdac845db406aefb181',
    local_dir='local_models/bge_m3'
)
```

Pipeline dùng AutoTokenizer/AutoModel cho embedding, AutoModelForSequenceClassification cho BGE reranker và AutoModelForCausalLM cho Qwen reranker. Loader không bật trust_remote_code. Phiên bản BGE-M3 chính thức này có pytorch_model.bin; PyTorch cần đủ mới cho chính sách đọc weights của Transformers. Notebook cài đúng các dependencies và giữ PyTorch phù hợp runtime CUDA.

Dense baseline Stage 8

CLS, dimension 1024, window max512/overlap32, FP16 inference/storage, L2 normalization và token-count-weighted aggregation. Không thêm query instruction hoặc ICD prefix. Đây là policy của vibiomir/dense_models.py và configs/dense_stage8.json.

BGE rerank S09 và dense recovery

Chấm logit một lớp cho cặp query/passage, FP16. Frozen policy S09 dùng max_pair_tokens=1024, query_window_tokens=384, overlap32, gộp mean theo query windows sau max theo passage windows. Các tham số thực đọc từ prepared identity; không thay bằng max4096 của notebook thử nghiệm mới.

BGE/Qwen trên submission và partial fallback

Hai loại notebook này dùng policy riêng tối đa4096 token/cặp, từ chối input vượt budget thay vì cắt ngầm. Fallback chấm chunk_text thống nhất cho evidence cũ/mới; corpus rerank đầy đủ chấm retrieval_text. Vì input khác, không reuse score chéo các route chỉ vì cùng model/chunk ID.

Qwen dùng instruction cố định:

```
Given a web search query, retrieve relevant passages that answer the query
```

Mẫu body:

```
<Instruct>: {instruction}
<Query>: {query}
<Document>: {passage}
```

Body được ghép với system/prefix/suffix trong scripts/qwen_submission_trial.py; lấy logit yes trừ no ở vị trí cuối. Không sampling, không sinh câu trả lời và không viết lại evidence. Để tái hiện đúng token template, dùng nguyên code thay vì tự tạo chat prompt tương đương.

Embedding đa nhánh thử nghiệm

Qwen dùng instruction query và last-token pooling; E5 dùng query:/passage: và mean pooling; BGE dùng CLS. Bản tối ưu mới chia window overlap64 và chọn BF16 chỉ khi native, nếu không dùng FP16. Các log cũ Qwen BF16/eager trên T4 thuộc cấu hình trước sửa; không nhầm policy đó với Dense V1 baseline hoặc gán nó cho toàn bộ artifact đã báo cáo.

4. Registry và khả năng tái hiện

Lưu model ID, revision tokenizer/weights, dtype, pooling/window, version torch/transformers/tokenizers, GPU, số parameter và hash code/config trong registry/plan. Checkpoint sau thay đổi policy phải tạo run mới. Khi chỉ cần kiểm chứng file bài nộp, khôi phục đúng packet/score đã bàn giao và xác minh hash; không cần inference lại toàn bộ model.

Cài đặt, vận hành và tái hiện bài nộp

1. Chọn đúng mức tái hiện

Bài đội chọn nộp: submission_17.zip. Hash local đã biết khớp REPORT export dense V1 trên Drive. Các nhánh Qwen/E5/corpus retrieval/fallback không phải thành phần của file này.

Mục tiêu	Cách làm
Kiểm tra đúng file đã nộp	Tải ZIP từ link, đối chiếu FINAL_SUBMISSION.json và audit schema
Tái xuất đúng kết quả đã có	Khôi phục prepared.zip, worker score và REPORT/EXPERIMENT của run dense V1
Chạy lại reranker từ candidate cũ	Notebook stage9_dense_recovery_colab.ipynb, khôi phục nguồn và Dense V1 candidates
Xây lại phương pháp từ raw data	Các lệnh phần 5; candidate output là contract native của repository, có thể khác bytes/thứ tự so với artifact Dense V1 lịch sử

Không sửa manifest producer để biến candidate native thành dense-results. Notebook recovery nhận producer lịch sử dense-results/dense-v2-cached-candidates; CLI dense_index.search xuất dense-candidates, được S09 nhận trực tiếp. Hai đường được giữ riêng. Muốn kiểm tra **đúng bài nộp theo byte**, dùng artifact lịch sử thay vì tự thay producer rồi yêu cầu hash giống hệt.

2. Môi trường và phụ thuộc

Linux hoặc WSL cho pipeline đầy đủ vì một số writer lock dùng fcntl. Python 3.10/3.11 là môi trường đề xuất cho cài lại; code có log đã chạy trên Colab Python 3.13 nhưng không coi mọi combination package là đã được kiểm thử.

Phần	Requirements / thư viện
Audit	requirements-eda.txt; PyArrow
Import/cleaning	requirements-stage3.txt, requirements-stage4.txt; PyArrow 22
Chunking	requirements-stage5.txt; Transformers4.57.1, tokenizers0.22.1, huggingface-hub0.35.3
Lexical cũ tùy chọn	requirements-stage7.txt; PyVi0.1.1, jieba0.42.1, NumPy
Dense/index	requirements-stage8.txt, requirements-stage8-colab.txt; NumPy, FAISS CPU1.12.0, PyTorch
S09	requirements-stage9.txt, requirements-stage9-colab.txt; Transformers4.57.1, tokenizers0.22.1, safetensors
Thử nghiệm mới	Dependencies được pin trong notebook; giữ PyTorch phù hợp GPU/driver

```
python -m venv .venv
source .venv/bin/activate
```

```
python -m pip install --upgrade pip
python -m pip install -r requirements-stage8.txt -r requirements-stage9.txt
python -m pip install -r requirements-stage8-colab.txt -r requirements-stage9-colab.txt
```

Lệnh GPU để pip chọn wheel phụ thuộc nền tảng; trên Colab giữ torch có sẵn nếu tương thích. Không cố dùng torch CUDA13 build cho driver không hỗ trợ. Lưu `python -m pip freeze`, `python --version`, `nvidia-smi` cùng run để ghi nhận môi trường thực tế.

3. Kiểm tra nhanh file private

Lấy file được đại diện xác nhận từ submission_17.zip trên Drive, đặt ở repository root. Tải query chính thức, đặt query/query.parquet hoặc Data/official/ViBioMIR/query.parquet.

```
python -m pip install pyarrow==22.0.0
python scripts/analyze_submission_zip.py --input submission_17.zip --query query/query.parquet --output
reports/submission17_reproduction_audit
```

Đối chiếu: 1.200 query đúng ID/thứ tự, 144.000 chunk, ZIP chỉ có submission.json; SHA256 ZIP `0963641cb422e4223d6fc4b0c22eb09938f8b03c8227053cc7a398bf33e25dbd`, JSON `24e4e6490afc53f2a08af229b54c354a7d47436b96f00f92efb1ebf6ba671e32`.

Audit ZIP không tự đo relevance hoặc đối chiếu toàn bộ corpus. Bằng chứng source validation của export gốc nằm tại evidence/REPORT.json và evidence/VALIDATION.json. Các cờ historical official_submission_ready=False / pending S11 được giữ nguyên; đây là trạng thái gate nội bộ lúc export, không được sửa thành chứng nhận của BTC.

4. Tái hiện từ artifact đã hoàn thành

4.1 Đường Colab

Tải/giữ cấu trúc Drive theo DU_LIEU_VA_ARTIFACT.md. Mở colab/stage9_dense_recovery_colab.ipynb trong runtime mới. Cấu hình chính: DENSE_VERSION=query_results_v1, RERANK_TOP_K=400, LIMITS=120/120/6/0.8. Chạy từng cell:

1. Mount Drive, tải requirements, giải nén **code bundle đã kiểm tra BUNDLE.json**.
2. Kiểm tra source sáu ZIP và 1.200 candidate files.
3. Restore source, kiểm tra query/links, adapt candidate bằng đối chiếu source.
4. Restore đúng prepared.zip nếu có; không rebuild gzip packet khi cần reuse score cũ.
5. Khôi phục worker theo namespace workers_by_packets có packet hash đúng. Nếu đã complete, không cần inference lại.
6. Nếu cần chấm lại, chạy pilot5 query, sau đó STOP_AFTER_QUERIES=None và rerun riêng worker cell.
7. Validate worker/source rồi package. Đối chiếu hash kết quả với bài gốc.

Run gốc: stage9_dense_query_results_v1_k400_120_120_v1. Export khớp hash: submission_20261031T065454786235Z.

4.2 Kiểm chứng bằng packet và score

Sau khi restore nguyên prepared bundle và score, các CLI của frozen producer tương ứng:

```
python -m vibiomir.stage9_pipeline validate-worker --prepared-dir PATH_TO_PREPARED --output-dir PATH_TO_WORKER_0 --worker-id 0
python -m vibiomir.stage9_pipeline export --prepared-dir PATH_TO_PREPARED --output-dir NEW_RESULTS --mode reranked --workers-dir
PATH_TO_WORKERS_PARENT
python -m vibiomir.stage9_pipeline validate --prepared-dir PATH_TO_PREPARED --output-dir NEW_RESULTS --workers-dir
PATH_TO_WORKERS_PARENT --chunk-dir PATH_TO_STRUCTURAL_STORE --queries Data/official/ViBioMIR/query.parquet
```

PATH_TO_WORKERS_PARENT phải chứa worker_0; thay các ký hiệu PATH bằng vị trí thật đã tải. Dùng đúng source modules của bundle tạo packet; không trộn PYTHONPATH với source khác phiên bản. Gzip timestamp có thể làm packet hash thay đổi dù văn bản giống; restore đúng bytes mới tái sử dụng score được.

5. Xây từ dữ liệu gốc

Các lệnh sau là đường rebuild phương pháp từ repository; chạy từ project root với data snapshot đúng và output chưa tồn tại. Các bước lớn có thể kéo dài nhiều ngày, cần local SSD/RAM và GPU. Nếu artifact đã đầy đủ thì ưu tiên phần 4 để kiểm tra bài gốc.

5.1 Audit và import version

```
python -m pip install -r requirements-eda.txt -r requirements-stage3.txt
python -m exploration.audit --data-dir Data --output-dir reports/data_audit/full_run_01 --batch-size 32 --sample-size 600
python -m vibiomir prepare-documents --data-dir Data --audit-dir reports/data_audit/full_run_01 --output-dir
artifacts/document_store_stage3_full_v2 --config configs/document_store_stage3.json
python -m vibiomir validate-documents --store-dir artifacts/document_store_stage3_full_v2 --data-dir Data
```

Data/ViBioMIR/shards phải chứa đúng raw shard của audit snapshot. Các input audit manifest/record_audit phải được cung cấp nếu bỏ qua audit. Hướng dẫn chi tiết và resume tại docs/DATA_AUDIT.md và docs/DOCUMENT_STORE_STAGE3.md.

5.2 Cleaning và structural chunks

```
python -m pip install -r requirements-stage5.txt
python -m vibiomir clean-documents --store-dir artifacts/document_store_stage3_full_v2 --data-dir Data --output-dir
artifacts/cleaning_stage4_full_apply --config configs/cleaning_stage4_reviewed.json --mode apply
python -m vibiomir validate-cleaned --output-dir artifacts/cleaning_stage4_full_apply --store-dir
artifacts/document_store_stage3_full_v2 --data-dir Data
python -m vibiomir prepare-chunk-inputs --cleaned-dir artifacts/cleaning_stage4_full_apply --store-dir
artifacts/document_store_stage3_full_v2 --data-dir Data --output-dir artifacts/stage5_inputs_full --batch-documents 500
python -m vibiomir download-chunk-tokenizer --output-dir artifacts/tokenizers/bge_m3_stage5
python -m vibiomir build-chunks --input-dir artifacts/stage5_inputs_full --output-dir artifacts/chunks_stage5_full_structure --
tokenizer-dir artifacts/tokenizers/bge_m3_stage5 --strategy structure --config configs/chunking_stage5.json
```

```
python -m vibiomir validate-chunks --input-dir artifacts/stage5_inputs_full --output-dir artifacts/chunks_stage5_full_structure --tokenizer-dir artifacts/tokenizers/bge_m3_stage5
```

Không dùng config cleaning chưa review để thay reviewed config. Bất kỳ source/config/model thay đổi nào đều tạo run khác; không kỳ vọng source manifest hash trùng nếu bytes hoặc môi trường producer khác. Xem docs/CLEANING_STAGE4.md và docs/CHUNKING_STAGE5.md để review policy/limitations.

5.3 BGE embedding, merge, chỉ mục và candidate native

```
python -m pip install -r requirements-stage8-colab.txt
python -m vibiomir.dense_stage8 plan --config configs/dense_stage8.json
python -m vibiomir.dense_stage8 bundle --config configs/dense_stage8.json --queries Data/official/ViBioMIR/query.parquet
```

Chạy colab/stage8_dense_worker.ipynb cho WORKER_ID=0..5, theo docs/RETRIEVAL_STAGES7_8.md. Có thể chạy tuần tự sáu logical worker nếu chỉ có một GPU. Worker0 tạo query_vectors. Sau đó merge và index:

```
python -m vibiomir.dense_stage8 merge --plan artifacts/dense_stage8_bge_structure/plan.json --chunk-dir artifacts/chunks_stage5_full_structure --workers-dir artifacts/dense_stage8_bge_structure --output-dir artifacts/dense_stage8_bge_structure/merged
python -m vibiomir.dense_index build --vector-dir artifacts/dense_stage8_bge_structure/merged --output-dir artifacts/dense_stage8_bge_structure/index_ivfpq
python -m vibiomir.dense_index search --vector-dir artifacts/dense_stage8_bge_structure/merged --index-dir artifacts/dense_stage8_bge_structure/index_ivfpq --query-dir artifacts/dense_stage8_bge_structure/query_vectors --output-dir artifacts/candidates_stage8_bge --top-k 500
```

Chỉ mục được train trên corpus vectors, không dùng relevance labels. Native candidate được đưa thẳng vào S09 ở bước sau. Producer Dense V1 lịch sử trên Drive có manifest/coverage và candidate đã có sẵn; code đóng gói không giả định việc đổi tên native output là đủ tái tạo producer đó.

5.4 BGE rerank top400 và xuất

configs/submission_17_rebuild.json là config CLI hợp lệ cho đường native candidate, cùng các giới hạn chính của phương pháp dense recovery; không thay thế prepared identity lịch sử.

```
python -m pip install -r requirements-stage9-colab.txt
python -m vibiomir.stage9_pipeline prepare --config configs/submission_17_rebuild.json
python -m vibiomir.stage9_pipeline rerank-worker --prepared-dir artifacts/retrieval_submission17_rebuild --output-dir artifacts/submission17_rebuild_workers/worker_0 --worker-id 0 --batch-size 8 --padded-tokens 8192
python -m vibiomir.stage9_pipeline export --prepared-dir artifacts/retrieval_submission17_rebuild --output-dir artifacts/submission17_rebuild_results --mode reranked --workers-dir artifacts/submission17_rebuild_workers
python -m vibiomir.stage9_pipeline validate --prepared-dir artifacts/retrieval_submission17_rebuild --output-dir artifacts/submission17_rebuild_results --workers-dir artifacts/submission17_rebuild_workers --chunk-dir artifacts/chunks_stage5_full_structure --queries Data/official/ViBioMIR/query.parquet
```

Để package với allowlist sau validation, gọi scripts/stage9_expand_support.package với results dir, danh sách ID query theo file official, tập ID integer từ official links và một thư mục output mới. Notebook dense recovery đã bao gồm bước package/đối chiếu links, là cách vận hành khuyến nghị cho việc này. Model inference và ANN có sai khác số học giữa môi trường; không bảo đảm hash rebuild bằng hash bài lịch sử.

6. Các route thử nghiệm khác

- Hybrid baseline cũ: configs/retrieval_stage9.json và docs/RETRIEVAL_STAGE9.md. Config gốc top200 và selection20/40/4, khác submission17.

- Chấm lại ZIP bằng Qwen/BGE: docs/COLAB_MULTIMODEL_EVIDENCE.md; không có retrieval mới.

- Bốn nhánh corpus mới: docs/COLAB_CORPUS_MULTIRETRIEVAL.md; chưa hoàn thành theo log nộp cho hồ sơ.

- Fallback: docs/COLAB_PARTIAL_RETRIEVAL_FALLBACK.md; nhập checkpoint dang dở và baseline, chấm theo thời gian rồi export đủ query. Không phải producer của submission17 được đại diện xác nhận.

7. Kiểm thử và ràng buộc tái hiện

```
python -m unittest discover -s tests -p 'test_stage9*.py' -v
python -m unittest tests.test_dense_recovery -v
python -m unittest tests.test_multimodel_evidence -v
python -m unittest tests.test_corpus_multiretrieval -v
python -m unittest tests.test_partial_retrieval_fallback.FallbackTest -v
```

Một số test tùy chọn cần đúng dependencies/tokenizer local; suite nguyên repository chưa được chạy lại toàn bộ khi đóng hồ sơ này. Hồ sơ giữ báo cáo lịch sử và test source, không gán một số test pass của phiên cũ cho toàn bộ code mới. Không có GT chính thức trong gói để tính F2/LCS tại local.

Nếu run báo thiếu nguồn, candidate producer khác, packet hash khác, hoặc thiếu query, dừng và dùng đúng artifact. Không sửa SHA/state để vượt kiểm tra. Dataset/API có thể cần cấp quyền phù hợp; mọi URL thực tế nằm ở LINKS.json.